Proposal Project Based Learning

Analisis Performa Protokol Komunikasi pada Sistem Monitoring Kolam Lele Berbasis IoT

Disusun Oleh:

ARIS NUR ROCHMAN	2407421063
DANIEL MARTIN GIDEON SIREGAR	2407421061
M RAFLI SAPUTRA	2407421085

Dosen Pembimbing:

Iik Muhamad Malik Matin, S.Kom., M.T.

NIP. 199408202022031009

PROGRAM STUDI TEKNIK MULTIMEDIA DAN JARINGAN

JURUSAN TEKNIK INFORMATIKA DAN KOMPUTER

POLITEKNIK NEGERI JAKARTA

2026

Lembar Persetujuan

Proposal Project Based Learning (PBL)

Analisis Performa Protokol Komunikasi pada Sistem Monitoring Kolam Lele Berbasis IoT

Telah disetujui untuk dipresentasikan di hadapan Tim Penilai *Project-Based Learning* kelas TMJ 5C, Jurusan Teknik Informatika dan Komputer, Politeknik Negeri Jakarta

Menyetujui
PIC Project



Iik Muhamad Malik Matin, S.Kom., M.T.
NIP. 199408202022031009

1. JUDUL PROYEK

Analisis Performa Protokol Komunikasi pada Sistem Monitoring Kolam Lele Berbasis IoT

2. RINGKASAN

Budidaya ikan lele memerlukan pemantauan kondisi air secara berkala agar perubahan pH, suhu, dan ketinggian air dapat segera diketahui. Dalam proyek ini, data sensor dikirim dari ESP32 ke server melalui protokol MQTT, HTTP, dan WebSocket pada sistem monitoring kolam lele berbasis Internet of Things (IoT). Ketiga protokol dibandingkan pada kondisi jaringan normal dan kondisi degradasi yang direkayasa secara terkontrol, dengan format serta ukuran payload, interval pengiriman, jumlah pesan, perangkat, dan durasi pengujian yang diseragamkan. Evaluasi Quality of Service (QoS) mengacu pada parameter *delay* (waktu tunda pengiriman), *jitter* (variasi waktu tunda), dan *packet loss* (persentase paket yang hilang) yang digunakan dalam pendekatan TIPHON. *Throughput* (laju data yang diterima) diukur sebagai metrik pendukung, sedangkan *response time*, *goodput*, dan *protocol overhead* dianalisis secara terpisah untuk menilai efisiensi komunikasi. Hasil pengujian diharapkan menunjukkan perubahan performa serta karakteristik setiap protokol pada kondisi jaringan yang berbeda.

3. KATA KUNCI

Kolam Lele, IoT (*Internet of Things*), MQTT, HTTP, WebSocket, QoS (*Quality of Service*), ESP32

4. PENDAHULUAN

4.1 Latar Belakang

Budidaya ikan lele merupakan salah satu kegiatan akuakultur air tawar yang keberhasilannya dipengaruhi oleh kondisi lingkungan budidaya, khususnya kualitas air. Perubahan parameter seperti suhu dan derajat keasaman (pH), serta perubahan ketinggian air dapat memengaruhi kondisi lingkungan kolam sehingga perlu dilakukan pemantauan secara berkala. Pada metode konvensional, pemantauan kondisi kolam umumnya masih bergantung pada pemeriksaan langsung sehingga perubahan kondisi air tidak selalu dapat diketahui secara cepat. Penelitian terkini pada budidaya *Clarias gariepinus* menunjukkan bahwa pemanfaatan sistem berbasis *Internet of Things* (IoT) memungkinkan parameter kualitas air seperti suhu dan pH

dipantau secara kontinu dan *real-time* [1]. Penelitian lain pada sistem akuakultur lele juga telah memanfaatkan berbagai parameter kualitas air sebagai data pemantauan berbasis IoT [2].

Internet of Things memungkinkan proses pemantauan dilakukan melalui integrasi antara sensor, mikrokontroler, jaringan komunikasi, dan server. Pada sistem monitoring kolam, sensor digunakan untuk memperoleh data kondisi air, kemudian mikrokontroler seperti ESP32 melakukan pembacaan dan pengiriman data melalui jaringan menuju server untuk disimpan, diolah, serta ditampilkan melalui sistem monitoring. Dengan pendekatan tersebut, kondisi kolam dapat diamati tanpa memerlukan pengukuran manual secara terus-menerus. Dalam proyek ini, parameter utama yang digunakan sebagai sumber data meliputi suhu, pH, dan ketinggian air. Ketiga parameter tersebut digunakan sebagai sumber data monitoring kondisi kolam, sedangkan fokus utama penelitian berada pada proses komunikasi jaringan yang membawa data sensor dari perangkat IoT menuju server.

Keberhasilan sistem monitoring berbasis IoT tidak hanya bergantung pada kemampuan sensor dalam memperoleh data, tetapi juga pada kualitas komunikasi jaringan yang digunakan untuk mentransmisikan data tersebut. Data sensor perlu dikirim dan diterima dengan tingkat keterlambatan, kestabilan waktu transmisi, dan tingkat keberhasilan pengiriman yang dapat dipertanggungjawabkan. Kondisi jaringan yang mengalami peningkatan *delay*, variasi waktu pengiriman (*jitter*), kehilangan paket (*packet loss*), maupun keterbatasan kapasitas jaringan dapat memengaruhi proses komunikasi antara perangkat IoT dan server. Oleh karena itu, evaluasi *Quality of Service* (QoS) menjadi salah satu aspek penting untuk mengetahui kualitas komunikasi suatu sistem IoT.

Dalam proyek ini, evaluasi QoS dilakukan dengan mengacu pada parameter kualitas jaringan yang digunakan dalam kerangka *Telecommunications and Internet Protocol Harmonization Over Networks* (TIPHON) [3]. Parameter utama yang dianalisis meliputi *delay*, *jitter* atau *delay variation*, dan *packet loss*. Nilai yang diperoleh dari hasil pengukuran digunakan untuk mengevaluasi perubahan kualitas jaringan pada setiap skenario pengujian. Selain parameter tersebut, *throughput* tetap digunakan sebagai metrik pendukung untuk mengetahui kemampuan jaringan dalam mentransmisikan data selama periode pengujian. Sementara itu, *response time*,

goodput, dan *protocol overhead* digunakan sebagai metrik tambahan untuk mengevaluasi karakteristik serta efisiensi komunikasi masing-masing protokol.

Beberapa protokol dapat digunakan untuk pertukaran data pada sistem IoT, di antaranya *Message Queuing Telemetry Transport* (MQTT), *Hypertext Transfer Protocol* (HTTP), dan *WebSocket*. Ketiganya memiliki mekanisme komunikasi yang berbeda. MQTT menggunakan model komunikasi *publish/subscribe* melalui sebuah *broker* dan dirancang sebagai protokol dengan kebutuhan sumber daya serta *network overhead* yang relatif kecil untuk lingkungan IoT dan *machine-to-machine communication* [4]. HTTP merupakan protokol aplikasi dengan pola komunikasi *request-response* yang bersifat *stateless* dan banyak digunakan pada layanan berbasis web maupun REST API [5]. Sementara itu, *WebSocket* menyediakan kanal komunikasi dua arah setelah proses *handshake*, sehingga *client* dan server dapat saling mengirim data melalui koneksi yang telah terbentuk [6]. Perbedaan mekanisme komunikasi tersebut dapat menghasilkan karakteristik lalu lintas dan performa jaringan yang berbeda ketika digunakan untuk membawa data sensor yang sama.

Penelitian sebelumnya oleh Sakinah telah melakukan implementasi dan analisis perbandingan *Quality of Service* antara MQTT, HTTP, CoAP, dan WebSocket pada sistem *Smart Pond* [7]. Penelitian tersebut menunjukkan bahwa masing-masing protokol memiliki karakteristik performa yang berbeda berdasarkan sejumlah parameter jaringan seperti *throughput*, *packet loss*, *delay*, *jitter*, dan *response time*. Namun, skenario pengujiannya lebih berfokus pada pengoperasian sistem *Smart Pond* dan pertukaran data pada kondisi pengujian yang digunakan dalam penelitian tersebut. Kondisi jaringan belum secara khusus direkayasa melalui perubahan karakteristik jaringan yang dikontrol untuk mengetahui respons masing-masing protokol terhadap degradasi kualitas jaringan.

Di sisi lain, penelitian mengenai performa protokol IoT pada kondisi jaringan yang mengalami gangguan menunjukkan bahwa perubahan kondisi jaringan dapat menghasilkan perubahan karakteristik QoS. Penelitian Nugroho, Pradana, dan Pamekas melakukan perbandingan MQTT dan CoAP pada jaringan intermiten dengan mengukur *delay*, *jitter*, dan *throughput*, serta menunjukkan adanya perubahan performa ketika kondisi jaringan terganggu [8]. Penelitian lain juga menganalisis mekanisme MQTT pada berbagai kondisi jaringan menggunakan parameter *latency*,

packet loss, *jitter*, dan penggunaan *bandwidth* [9]. Hal tersebut menunjukkan bahwa performa protokol komunikasi tidak cukup hanya diamati pada kondisi jaringan normal, tetapi perlu dievaluasi pula pada kondisi jaringan yang mengalami degradasi.

Berdasarkan permasalahan tersebut, proyek ini mengembangkan sistem monitoring kolam lele berbasis IoT sekaligus lingkungan pengujian jaringan untuk menganalisis kinerja protokol MQTT, HTTP, dan WebSocket. Ketiga protokol digunakan untuk mentransmisikan data sensor dengan kondisi eksperimen yang dibuat setara, meliputi format dan ukuran *payload*, interval pengiriman, jumlah pesan, perangkat keras, infrastruktur jaringan, server, serta durasi pengujian. Kondisi jaringan kemudian direkayasa secara terkontrol melalui Linux Gateway untuk menghasilkan beberapa kondisi pengujian sehingga perubahan performa setiap protokol dapat diamati secara konsisten.

Lalu lintas jaringan pada setiap skenario direkam menggunakan Wireshark untuk memperoleh data pengukuran. Hasil pengujian selanjutnya dianalisis berdasarkan parameter QoS yang mengacu pada TIPHON, terutama *delay*, *jitter*, dan *packet loss*, sedangkan *throughput* digunakan sebagai metrik performa jaringan pendukung. Selain itu, *response time*, *goodput*, dan *protocol overhead* dianalisis secara terpisah sebagai metrik tambahan untuk memberikan gambaran mengenai efisiensi komunikasi MQTT, HTTP, dan WebSocket. Pemisahan tersebut dilakukan agar parameter QoS yang digunakan sebagai acuan penilaian tidak bercampur dengan parameter efisiensi pada tingkat aplikasi dan protokol.

Dengan demikian, sistem monitoring kolam lele dalam proyek ini berfungsi sebagai studi kasus implementasi *Internet of Things*, sedangkan fokus utama proyek berada pada analisis performa komunikasi jaringan. Hasil penelitian diharapkan dapat memberikan gambaran mengenai karakteristik QoS dan efisiensi komunikasi MQTT, HTTP, dan WebSocket pada kondisi jaringan yang berbeda. Hasil tersebut selanjutnya dapat digunakan untuk memahami perubahan performa serta *trade-off* masing-masing protokol berdasarkan karakteristik jaringan yang dihadapi.

4.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam proyek ini adalah sebagai berikut:

1. Bagaimana pembuatan sistem monitoring kolam lele berbasis Internet of Things menggunakan ESP32, sensor suhu, sensor pH, dan water level sensor sebagai sumber data kondisi kolam?
2. Bagaimana perancangan arsitektur jaringan dan lingkungan pengujian yang menghubungkan node ESP32, Router, Linux Gateway, dan server untuk mendukung pengiriman serta pengujian data sensor?
3. Bagaimana implementasi protokol MQTT, HTTP, dan WebSocket pada sistem monitoring kolam lele untuk mengirimkan data sensor dari ESP32 menuju server dengan kondisi komunikasi yang setara?
4. Bagaimana hasil perbandingan performa MQTT, HTTP, dan WebSocket pada kondisi jaringan normal dan kondisi jaringan yang mengalami degradasi secara terkontrol berdasarkan parameter QoS yang mengacu pada TIPHON serta metrik efisiensi komunikasi?

4.3 *State of the Art* dan Kebaruan

Berbagai penelitian telah dilakukan terkait pengembangan sistem monitoring akuakultur berbasis Internet of Things (IoT) maupun evaluasi performa protokol komunikasi pada jaringan IoT. Penelitian-penelitian tersebut menggunakan objek, protokol, parameter pengujian, dan kondisi jaringan yang berbeda. Ringkasan penelitian terdahulu yang relevan dengan proyek ini ditunjukkan pada tabel berikut.

No.	Penelitian	Fokus	Metode/Karakteristik	Keterbatasan terhadap Proyek
1.	Sakinah, 2024 [7]	Smart Pond dan perbandingan protokol IoT	Membandingkan MQTT, HTTP, CoAP, dan WebSocket berdasarkan parameter QoS	Hasil pengujian hanya ditampilkan pada platform telegram dan tidak dilakukan rekayasa pengujian
2.	Pramartaningthyas *et al.*, 2026 [1]	Monitoring dan kontrol kualitas air *Clarias gariepinus*	Monitoring suhu dan pH berbasis IoT	Tidak berfokus pada perbandingan performa beberapa protokol komunikasi maupun karakteristik trafik jaringan
3.	Yuusufa *et al.*, 2026 [2]	Monitoring akuakultur lele	Monitoring suhu, pH, DO, dan amonia dengan komunikasi	Menggunakan satu protokol komunikasi sehingga tidak

No.	Penelitian	Fokus	Metode/Karakteristik	Keterbatasan terhadap Proyek
			MQTT	mengevaluasi perbedaan karakteristik antar protokol
4.	Nugroho *et al.*, 2026 [8]	Perbandingan protokol pada jaringan intermiten	Membandingkan MQTT dan CoAP berdasarkan delay, jitter, dan throughput	Tidak membandingkan HTTP dan WebSocket serta tidak diterapkan pada sistem Smart Pond
5.	Savchenko dan Klymenko, 2026 [9]	Analisis performa MQTT pada jaringan IoT	Pengujian latency, packet loss, jitter, bandwidth, dan mekanisme QoS MQTT	Berfokus pada karakteristik MQTT dan tidak melakukan perbandingan dengan HTTP dan WebSocket
6.	Proyek yang diusulkan	Sistem monitoring kolam lele dan analisis performa jaringan IoT	Perbandingan MQTT, HTTP, dan WebSocket pada kondisi jaringan terkontrol dengan evaluasi QoS yang mengacu pada TIPHON serta analisis efisiensi komunikasi	Mengevaluasi perubahan performa ketiga protokol terhadap degradasi jaringan dengan kondisi eksperimen yang diseragamkan serta membandingkan hasil QoS dan efisiensi komunikasinya

Tabel 4.1 Penelitian Terdahulu

Berdasarkan penelitian terdahulu, penggunaan IoT untuk monitoring kualitas air, implementasi MQTT, HTTP, dan WebSocket, maupun pengukuran parameter Quality of Service bukan merupakan hal baru secara individual. Penggunaan ESP32, sensor kualitas air, maupun penerapan parameter QoS yang mengacu pada TIPHON juga tidak ditempatkan sebagai unsur kebaruan utama dalam proyek ini.

Kesenjangan penelitian yang menjadi dasar proyek ini terletak pada belum terintegrasinya secara spesifik perbandingan performa MQTT, HTTP, dan WebSocket pada sistem monitoring kolam dengan pengujian kondisi jaringan yang direkayasa

secara terkontrol. Pada proyek ini, performa ketiga protokol tidak hanya diamati pada kondisi jaringan normal, tetapi juga pada kondisi degradasi jaringan untuk mengetahui perubahan karakteristik komunikasi masing-masing protokol.

Kontribusi utama proyek ini terletak pada rancangan eksperimen komparatif yang mengevaluasi perubahan performa MQTT, HTTP, dan WebSocket pada kondisi jaringan yang berbeda. Pengujian dilakukan menggunakan format dan ukuran payload, interval transmisi, jumlah pesan, perangkat keras, infrastruktur jaringan, server, dan durasi pengujian yang dikontrol agar setiap protokol memperoleh kondisi eksperimen yang setara. Hasil pada setiap kondisi pengujian kemudian dibandingkan terhadap kondisi baseline untuk mengetahui perubahan performa relatif masing-masing protokol.

Evaluasi QoS dilakukan dengan mengacu pada parameter TIPHON, khususnya delay, jitter atau delay variation, dan packet loss. Throughput digunakan sebagai metrik pendukung performa jaringan. Selain itu, response time, *goodput*, dan *protocol overhead* dianalisis secara terpisah sebagai metrik tambahan untuk mengevaluasi efisiensi komunikasi masing-masing protokol.

Dengan pendekatan tersebut, kebaruan proyek ini tidak terletak pada penggunaan TIPHON, ESP32, maupun protokol MQTT, HTTP, dan WebSocket secara individual, tetapi pada integrasi pengujian komparatif ketiga protokol dalam lingkungan jaringan yang dikontrol, evaluasi perubahan performa terhadap kondisi baseline, serta pemisahan antara evaluasi QoS berbasis TIPHON dan analisis efisiensi komunikasi. Hasil penelitian diharapkan dapat memberikan gambaran mengenai karakteristik dan trade-off MQTT, HTTP, dan WebSocket pada kondisi kualitas jaringan yang berbeda.

4.4 Tujuan dan Manfaat Proyek

4.4.1 Tujuan

Tujuan yang ingin dicapai dalam pelaksanaan proyek ini adalah:

a. Merancang dan mengimplementasikan prototipe sistem monitoring kolam lele berbasis *Internet of Things* menggunakan sensor kualitas air dan mikrokontroler ESP32.

b. Mengimplementasikan protokol MQTT, HTTP, dan WebSocket sebagai mekanisme komunikasi antara perangkat IoT dan server.
c. Merancang lingkungan pengujian jaringan yang memungkinkan rekayasa kondisi jaringan secara terkontrol untuk mengevaluasi perubahan performa masing-masing protokol komunikasi.
d. Mengukur dan mengevaluasi *Quality of Service* MQTT, HTTP, dan WebSocket berdasarkan parameter yang mengacu pada TIPHON, meliputi *delay*, *jitter*, dan *packet loss*, serta menggunakan *throughput* sebagai metrik pendukung performa jaringan.
e. Menganalisis *response time*, *goodput*, dan *protocol overhead* sebagai metrik tambahan untuk mengevaluasi efisiensi komunikasi MQTT, HTTP, dan WebSocket.
f. Menganalisis pengaruh perubahan kondisi jaringan terhadap karakteristik QoS dan efisiensi komunikasi masing-masing protokol.
g. Membandingkan karakteristik dan *trade-off* MQTT, HTTP, dan WebSocket berdasarkan hasil pengujian pada kondisi jaringan yang berbeda.

4.4.2 Manfaat

a. Manfaat Akademis

Proyek ini diharapkan dapat menjadi referensi dalam implementasi dan analisis jaringan pada sistem *Internet of Things*, khususnya mengenai karakteristik QoS protokol MQTT, HTTP, dan WebSocket pada kondisi jaringan yang berbeda. Hasil pengujian diharapkan dapat memberikan data eksperimental mengenai parameter QoS yang mengacu pada TIPHON serta karakteristik efisiensi komunikasi masing-masing protokol.

b. Manfaat Praktis

Prototipe yang dihasilkan dapat menjadi dasar pengembangan sistem monitoring kondisi kolam lele yang mampu mengirimkan data sensor secara *real-time*. Hasil analisis dapat digunakan sebagai bahan pertimbangan dalam memahami karakteristik MQTT, HTTP, dan WebSocket sesuai dengan kebutuhan komunikasi serta kondisi jaringan tempat sistem IoT diterapkan.

c. Manfaat Teknis

Dari sisi jaringan, proyek ini diharapkan dapat memberikan gambaran mengenai perubahan *delay*, *jitter*, *packet loss*, dan *throughput* pada MQTT, HTTP, dan WebSocket ketika kondisi jaringan mengalami degradasi. Selain itu, analisis *response time*, *goodput*, dan *protocol overhead* diharapkan dapat memberikan gambaran mengenai efisiensi komunikasi masing-masing protokol. Hasil tersebut dapat digunakan untuk memahami karakteristik dan *trade-off* ketiga protokol pada berbagai kondisi jaringan.

5. RUANG LINGKUP PROYEK

5.1 Lingkup Proyek

Ruang lingkup proyek ini mencakup perancangan, implementasi, dan pengujian sistem monitoring kolam lele berbasis Internet of Things (IoT) dengan fokus utama pada analisis performa komunikasi jaringan. Sistem dirancang menggunakan mikrokontroler ESP32 sebagai perangkat pengumpul dan pengirim data sensor menuju server melalui jaringan Wi-Fi. Adapun lingkup pelaksanaan proyek meliputi:

1. Merancang dan mengimplementasikan sistem monitoring kolam lele berbasis *Internet of Things* (IoT) menggunakan mikrokontroler ESP32.
2. Menggunakan tiga sensor utama berupa sensor suhu, sensor pH, dan *water level sensor* atau sensor ketinggian air.
3. Mengimplementasikan tiga protokol komunikasi, yaitu MQTT, HTTP, dan WebSocket.
4. Menggunakan jaringan Wi-Fi sebagai media komunikasi antara perangkat IoT dan infrastruktur jaringan.
5. Menggunakan server sebagai *endpoint* penerima data dari protokol MQTT, HTTP, dan WebSocket.
6. Menggunakan format dan ukuran *payload*, interval pengiriman, jumlah pesan, perangkat keras, serta durasi pengujian yang diseragamkan pada setiap protokol agar proses perbandingan dilakukan dalam kondisi yang setara.
7. Merancang lingkungan pengujian jaringan yang terdiri atas ESP32 sebagai node IoT, router, Linux Gateway berbasis Raspberry Pi, dan server.

8. Melakukan rekayasa kondisi jaringan (*network impairment*) berupa kondisi jaringan normal (*baseline*), penambahan *latency*, *packet loss*, dan pembatasan *bandwidth* secara terkontrol.
9. Menggunakan tc/netem atau mekanisme *traffic control* sejenis pada Linux Gateway untuk membentuk kondisi jaringan sesuai dengan skenario pengujian.
10. Melakukan *packet capture* dan analisis lalu lintas jaringan menggunakan Wireshark.
11. Mengukur dan mengevaluasi *Quality of Service* (QoS) berdasarkan parameter yang mengacu pada TIPHON, yaitu *delay*, *jitter* atau *delay variation*, dan *packet loss*.
12. Mengukur *throughput* sebagai metrik performa jaringan pendukung untuk mengetahui laju data yang berhasil ditransmisikan selama proses pengujian.
13. Menganalisis *response time*, *goodput*, dan *protocol overhead* sebagai metrik tambahan untuk mengevaluasi efisiensi komunikasi MQTT, HTTP, dan WebSocket.
14. Membandingkan karakteristik performa MQTT, HTTP, dan WebSocket pada setiap kondisi jaringan yang diuji.
15. Mengimplementasikan basis data dan dashboard sederhana untuk menerima, menyimpan, serta menampilkan data sensor sebagai bagian dari sistem monitoring.
16. Melakukan analisis terhadap perubahan performa dan *trade-off* MQTT, HTTP, dan WebSocket berdasarkan hasil pengukuran QoS, performa jaringan, dan efisiensi komunikasi.

5.2 Batasan Proyek

Agar pelaksanaan proyek tetap terarah dan sesuai dengan fokus *Project Based Learning* pada analisis jaringan, batasan proyek ditentukan sebagai berikut:

1. Fokus utama proyek adalah analisis performa komunikasi jaringan pada sistem IoT, sedangkan sistem monitoring kolam lele digunakan sebagai studi kasus implementasi.
2. Protokol komunikasi yang dibandingkan dibatasi pada MQTT, HTTP, dan WebSocket.
3. Sensor digunakan sebagai sumber data pengujian dan tidak menjadi objek utama analisis penelitian.

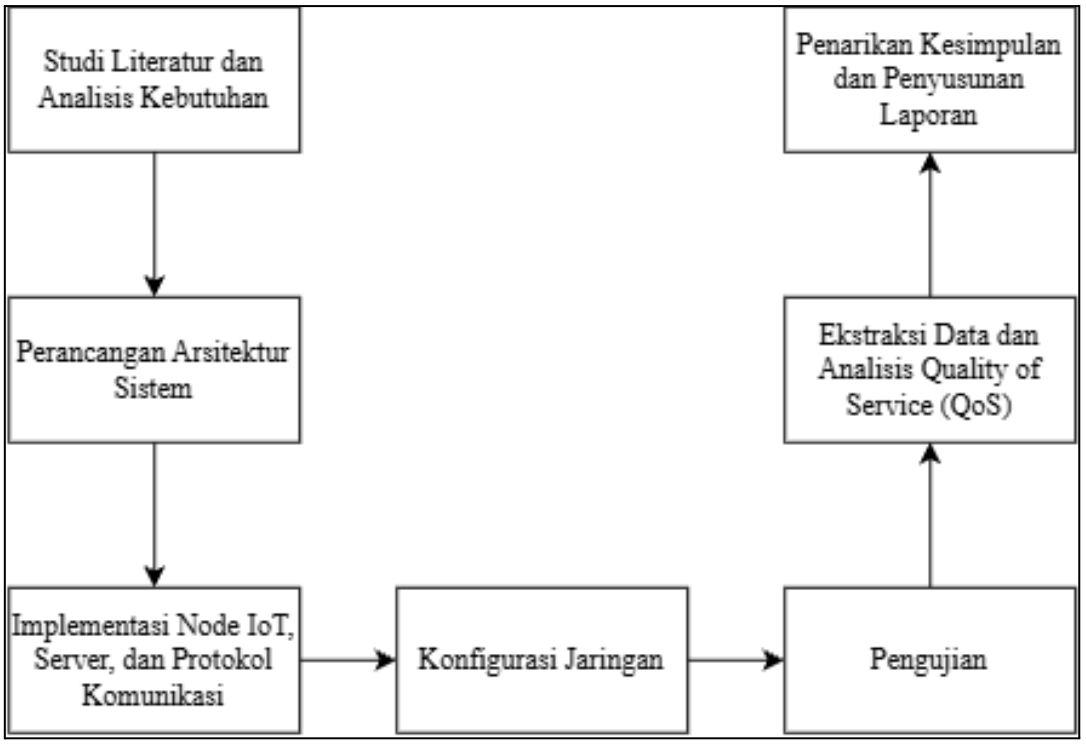
4. Parameter kondisi kolam yang digunakan sebagai sumber data dibatasi pada suhu, pH, dan ketinggian air.
5. Komunikasi perangkat IoT dibatasi menggunakan jaringan Wi-Fi berbasis TCP/IP.
6. Teknologi komunikasi lain seperti LoRa, Zigbee, NB-IoT, jaringan seluler, dan teknologi komunikasi sejenis tidak menjadi bagian dari pengujian utama.
7. Pengujian dilakukan pada lingkungan jaringan yang dikontrol agar kondisi eksperimen dapat direproduksi.
8. Rekayasa kondisi jaringan dibatasi pada penambahan *latency*, *packet loss*, dan pembatasan *bandwidth*, dengan kondisi jaringan normal digunakan sebagai *baseline*.
9. Setiap jenis *network impairment* pada pengujian utama diterapkan secara terpisah untuk mengetahui pengaruh masing-masing kondisi terhadap performa protokol.
10. Format dan ukuran *payload*, interval pengiriman, jumlah pesan, perangkat keras, infrastruktur jaringan, server, serta durasi pengujian dikontrol agar tetap sama pada setiap protokol.
11. Konfigurasi masing-masing protokol dipertahankan tetap selama seluruh skenario pengujian agar perubahan hasil tidak disebabkan oleh perubahan konfigurasi aplikasi.
12. Evaluasi *Quality of Service* yang mengacu pada TIPHON dibatasi pada parameter *delay*, *jitter* atau *delay variation*, dan *packet loss*.
13. *Throughput* digunakan sebagai metrik performa jaringan pendukung dan dianalisis secara terpisah dari parameter utama QoS yang mengacu pada TIPHON.
14. *Response time*, *goodput*, dan *protocol overhead* digunakan sebagai metrik tambahan untuk mengevaluasi efisiensi komunikasi MQTT, HTTP, dan WebSocket.
15. *Machine Learning* tidak digunakan dalam proses analisis data.
16. Aspek performa sensor, konsumsi daya perangkat, penggunaan CPU dan memori, serta aspek lain di luar performa komunikasi jaringan tidak menjadi fokus utama pengujian.
17. Analisis hasil difokuskan pada karakteristik QoS, performa jaringan, efisiensi komunikasi, serta perubahan performa MQTT, HTTP, dan WebSocket pada kondisi jaringan yang berbeda.

18. Fitur keamanan dibatasi hanya menggunakan algoritma AES untuk keamanan isi pesan/paket dan protokol TLS untuk keamanan jalur pengirimannya.

6. METODOLOGI PELAKSANAAN

6.1 Diagram Alir Percobaan

Dilakukannya percobaan ini untuk membandingkan proses pengiriman data dari 3 protokol berbeda yaitu MQTT, HTTP, dan WebSocket. Dalam prosesnya terdapat beberapa tahapan yang dapat dilakukan untuk menjawab permasalahan ini



Gambar 6.1 Diagram Alir Percobaan

6.1.1 Studi Literatur dan Analisis Kebutuhan

Tahap pengumpulan referensi akademik yang relevan dengan protokol MQTT, HTTP, WebSocket, serta standar pengukuran Quality of Service (QoS). Pada tahap ini juga dilakukan identifikasi kebutuhan perangkat keras berupa mikrokontroler ESP32, sensor suhu, sensor pH, dan water level sensor, serta perangkat lunak yang dibutuhkan untuk membangun lingkungan proyek nantinya termasuk diantaranya protokol dan algoritma sekuriti yang digunakan, hingga alat yang digunakan dalam pengambilan data nantinya.

Selain itu pada tahap ini juga ditetapkan pH dan suhu air sesuai dengan ketentuan dari Kementerian Kelautan dan Perikanan untuk budidaya lele, dimana kondisi pH berada pada tingkat 7-8 dan suhu berada pada 25-30°C. Penentuan *baseline* pH dan suhu diperlukan agar dalam pembuatan sistem monitoring dapat diketahui batasan *alert* yang akan diinformasikan pada pengguna.

6.1.2 Perancangan Arsitektur Sistem

Merancang topologi jaringan yang memetakan posisi node sensor, titik router, Linux Gateway, hingga server tujuan. Perancangan ini juga mendefinisikan letak titik penyadapan lalu lintas jaringan (*packet capture point*) agar data lalu lintas yang direkam representatif terhadap kondisi aktual transmisi data. Selain itu, pada tahap perancangan juga ditentukan mekanisme pengamanan komunikasi yang ada diterapkan pada masing-masing protokol yang digunakan.

MQTT diamankan menggunakan MQTT *over* TLS (MQTTS) serta mekanisme autentikasi dan ACL berdasarkan topik untuk membatasi hak *publish* dan *subscribe* setiap *client*. HTTP diamankan menggunakan HTTPS dengan mekanisme autentikasi berupa API key atau token untuk membatasi akses ke endpoint server. Sementara itu, WebSocket diamankan menggunakan WebSocket *Secure* (WSS) dengan autentikasi berbasis token atau API key pada proses pembentukan koneksi. Penerapan mekanisme tersebut bertujuan untuk menjaga kerahasiaan data selama transmisi berlangsung, memverifikasi identitas *client*, serta membatasi akses yang dimiliki *client* terhadap layanan yang tersedia.

Selain aspek keamanan/*security* pada protokol komunikasi, juga dilakukan aspek keamanan pada sisi paket data dan aplikasi. Pada hasil data dilakukan enkripsi dengan algoritma AES sehingga bentuk data yang dikirimkan tidak dalam bentuk string mentah melainkan berbentuk *ciphertext*. Kemudian pada website monitoring juga diterapkan aspek keamanan melalui autentikasi *username* dan *password*. Kegunaan autentikasi ini nantinya untuk mencegah kebocoran data monitoring sehingga hanya pengguna tertentu saja yang dapat mengakses data monitoring.

6.1.3 Implementasi Node IoT, Server, dan Protokol Komunikasi

Merakit komponen perangkat keras menjadi sebuah *prototype* dan mengembangkan kode program pada ESP32. Tahap ini berfokus pada implementasi mekanisme pengiriman pesan menggunakan protokol MQTT, HTTP, dan WebSocket agar dapat berjalan secara stabil dalam mengirimkan data sensor ke *backend* server.

Pada tahap ini juga dikembangkan kode program untuk website monitoring. Pengembangan tersebut mulai dari *frontend* hingga *backend* termasuk disana *database* untuk menerima paket data dari masing-masing node ESP32 dengan protokol yang berbeda-beda.

6.1.4 Konfigurasi Jaringan

Pada tahap ini mulai diatur untuk konfigurasi jaringan mulai dari router hingga *end device* seperti ESP32 hingga server. Selain itu ditahap ini dibangun pula lingkungan jaringan yang dapat dikontrol menggunakan mesin Linux yang bertindak sebagai *gateway*. Konfigurasi dilakukan memanfaatkan *traffic control* (tc) dan netem untuk menginjeksi degradasi jaringan buatan seperti penambahan *latency*, persentase *packet loss*, dan pembatasan kapasitas *bandwidth*.

6.1.5 Pengujian

Pada tahap ini pertama-tama dilakukan penyamaan parameter, meliputi kesamaan ukuran *payload* aplikasi, kesamaan jumlah pesan yang dikirim, dan kesamaan interval waktu pengiriman untuk menjaga keadilan dalam eksperimen.

Setelah penyamaan, maka dilakukan eksekusi pengiriman data sensor dari node IoT ke server melalui jaringan tanpa rekayasa degradasi terlebih dahulu. Lalu lintas jaringan direkam secara menyeluruh menggunakan Wireshark untuk mendapatkan nilai performa dasar (*baseline*) dari masing-masing protokol.

Setelah performa dasar didapatkan maka dilakukan eksekusi pengiriman data namun dengan rekayasa degradasi terkontrol yaitu menerapkan profil *network impairment* yang telah dikonfigurasi pada Linux Gateway secara bergantian (berdasarkan skenario *latency*, *packet loss*, dan limitasi *bandwidth*).

Setelah diberikan *network impairment*, dilakukan *packet capture* pada setiap uji coba untuk diekstrak dan dianalisa pada tahap berikutnya.

6.1.6 Ekstraksi Data dan Analisis Quality of Service

Memproses arsip tangkapan jaringan (*PCAP files*) untuk mengekstraksi parameter metrik yang telah ditentukan. Analisis komparatif dilakukan dengan membandingkan nilai *throughput*, *delay*, *jitter*, *packet loss*, *response time*, *goodput*, dan *protocol overhead* dari ketiga protokol terhadap perubahan kondisi jaringan.

6.1.7 Penarikan Kesimpulan dan Penyusunan Laporan

Mensintesis hasil analisis statistik menjadi kesimpulan teknis mengenai karakteristik performa dan efisiensi komunikasi dari MQTT, HTTP, dan WebSocket. Tahap akhir mencakup pendokumentasian seluruh proses, metode, dan hasil pengujian ke dalam laporan akhir *Project Based Learning* (PBL).

6.2 Analisis Kebutuhan Sistem dan Jaringan

Kebutuhan sistem dan jaringan untuk implementasi proyek ini dibagi menjadi dua kategori, yaitu perangkat keras dan perangkat lunak:

6.2.1 Kebutuhan Perangkat Keras

a. Mikrokontroler ESP32: Berfungsi sebagai *node* IoT utama untuk mengakuisisi data dari sensor dan mentransmisikannya ke jaringan menggunakan modul Wi-Fi bawaan [10], [11], [12].

b. Sensor Suhu DS18B20: Komponen sensor digital untuk memantau suhu air tambak secara presisi menggunakan antarmuka 1-Wire [10], [13].
c. Sensor pH: Perangkat keras yang digunakan untuk membaca tingkat keasaman atau kebasaan air kolam budidaya [10], [13].
d. Water Level Sensor: Perangkat keras yang digunakan untuk mengukur ketinggian air pada kolam sebagai salah satu parameter monitoring kondisi kolam.
e. Router Wi-Fi: Infrastruktur jaringan nirkabel lokal pada pita 2,4 GHz yang menjadi penghubung komunikasi antara ESP32 dan lingkungan pengujian [14], [12].
f. Server (*Backend*): Komputer terpusat yang difungsikan untuk menerima paket data aplikasi, menjalankan layanan *broker/server* protokol, serta mencatat lalu lintas komunikasi [14], [12].

6.2.2 Kebutuhan Perangkat Lunak

a. Utilitas tc (*Traffic Control*) dan netem: Perangkat lunak subsistem Linux yang digunakan untuk merekayasa kondisi jaringan secara presisi, meliputi penambahan *latency*, *packet loss*, dan pembatasan *bandwidth* (*bandwidth limitation*)
b. Wireshark: Perangkat lunak penganalisis jaringan (*packet analyzer*) yang digunakan untuk merekam seluruh sesi transmisi (*packet capture*) guna keperluan ekstraksi dan perhitungan parameter QoS serta *protocol overhead* [14].
c. MQTT *Broker* (contoh: Mosquitto): Layanan perantara yang memfasilitasi komunikasi pesan berbasis *publish-subscribe* antara ESP32 dan server [4], [14], [12].
d. HTTP Server & WebSocket Server: Lingkungan perangkat lunak *backend* (misalnya berbasis Node.js, FastAPI, atau Flask) untuk memproses lalu lintas *request-response* HTTP dan koneksi persisten dua arah WebSocket [5], [6], [14].
e. Sistem Basis Data (contoh: PostgreSQL / InfluxDB): Perangkat lunak untuk mengarsipkan data sensor dan parameter uji jaringan secara terstruktur.

f. Aplikasi *Dashboard* (contoh: Grafana): Sistem antarmuka visual berbasis web untuk menampilkan pemantauan data sensor secara interaktif [10], [13].

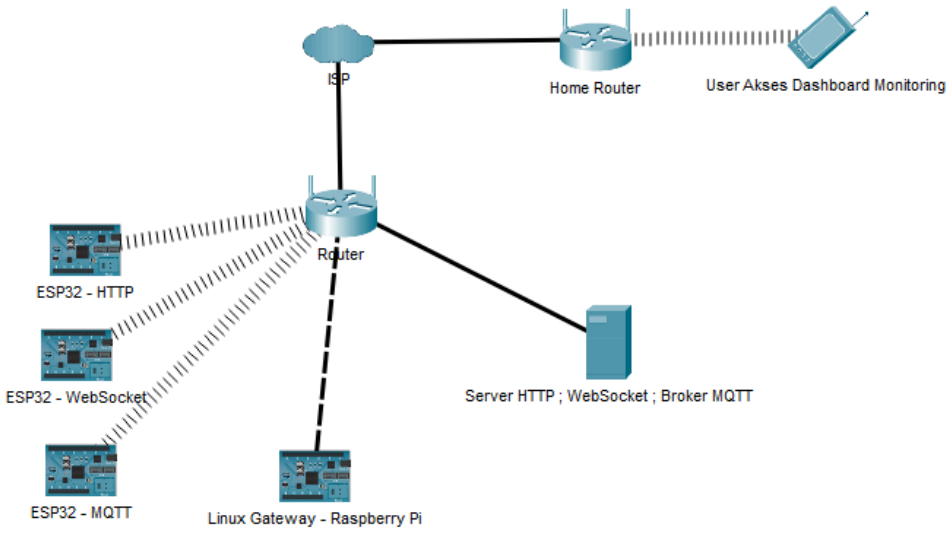
g. Arduino IDE / PlatformIO: *Integrated Development Environment* untuk menulis, brush mengkompilasi, dan mengunggah program pengiriman data (C/C++) beserta pustaka pendukung protokol ke modul ESP32 [10], [13].

6.3 Desain dan Arsitektur Pengujian:

Desain dan arsitektur pengujian meliputi desain dan arsitektur jaringan, Node IoT, dan Server [14]. Dalam subbab ini juga meliputi gambaran topologi dan *flow* arsitektur pengujian.

6.3.1 Desain jaringan mencakup:

a. Topologi Jaringan



Gambar 6.2 Desain Topologi Jaringan

Pada topologi tersebut divisualisasikan rangkaian konektivitas antar *ende devices*. Untuk jenis topologi yang digunakan merupakan topologi tree. Topologi ini tersusun dari 2 topologi inti yaitu topologi star yaitu antara router dengan *end devices* seperti Node ESP32 hingga server dan juga topologi line yaitu antara router dengan Internet Service Provider (ISP) hingga menuju perangkat *smartphone* yang bisa digunakan untuk memonitoring data sensor.

b. Protokol Jaringan:

Untuk protokol jaringan yang digunakan adalah IPv4 untuk pengalamatan jaringan, ARP (Address Resolution Protocol) untuk mencocokkan alamat IP dengan alamat MAC fisik perangkat di jaringan lokal, serta NAT (Network Address Translation) untuk mencocokkan alamat IP privat jaringan lokal dengan IP publik dari ISP agar perangkat di jaringan lokal dapat mengakses internet. Sedangkan untuk pembagian alamat IP digunakan protokol *static*.

c. Firewall:

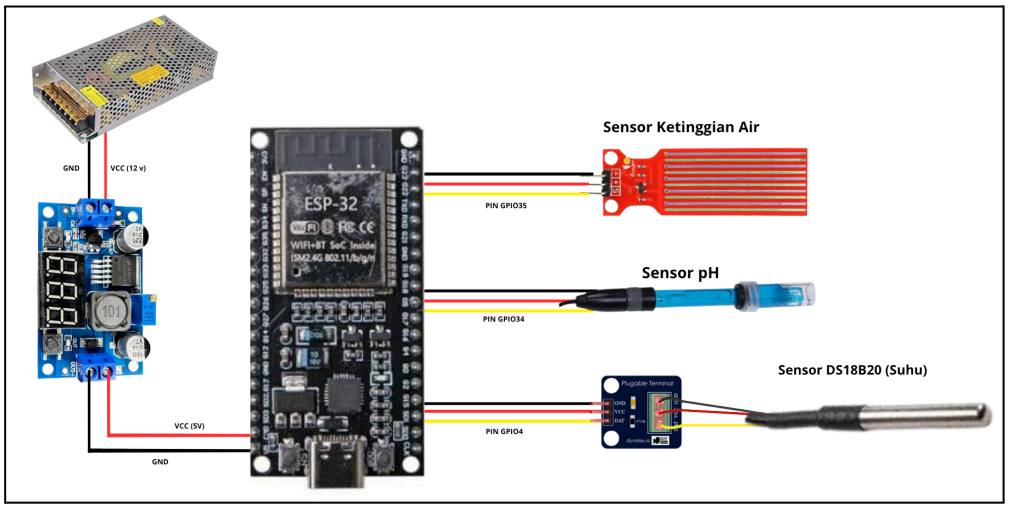
Untuk konfigurasi *firewall* dilakukan konfigurasi dasar pada router yaitu dengan NAT *Firewall* dengan protokol Masquerade.

d. Alamat IP:

Karena pembagian alamat IP dilakukan dalam bentuk static, maka pembagiannya sebagai berikut:

1. Router: (Dari ISP)
2. Linux Gateway (Raspberry Pi): 192.168.20.2/24
3. ESP32 - MQTT: 192.168.10.2/24
4. ESP32 - HTTP: 192.168.10.3/24
5. ESP32 - WebSocket: 192.168.10.4/24
6. Server: 192.168.30.2/24

6.3.2 Desain Node IoT

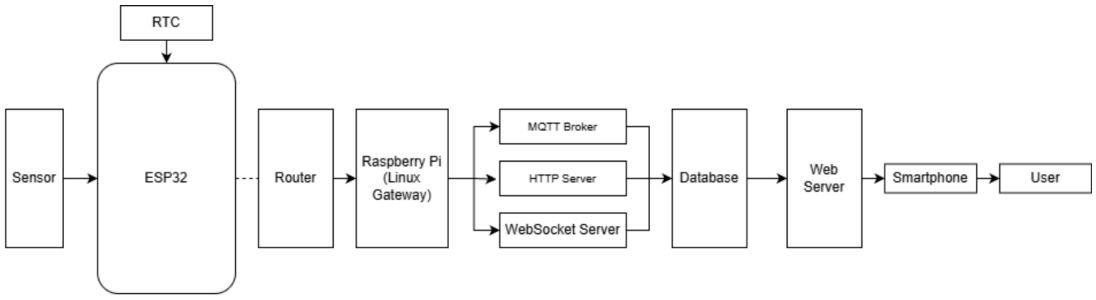


Gambar 6.3 Desain Topologi Node IoT

Pada topologi tersebut, setiap Node ESP32 terdiri dari sebuah ESP32, sebuah sensor pH, sebuah sensor ketinggian air, dan sebuah sensor suhu. Untuk sumber listrik bertegangan 5V dihasilkan dari *power supply* 12V dan *step down* DC *to* DC *adjustable* menuju tegangan 5V.

Selain itu masing-masing sensor terkoneksi melalui pin-pin pada ESP32. Sedangkan untuk penggunaan pinnya sendiri yaitu pin analog 35 untuk sensor ketinggian air, pin analog 34 untuk sensor pH, dan pin digital 4 untuk sensor suhu.

6.3.3 Desain Server



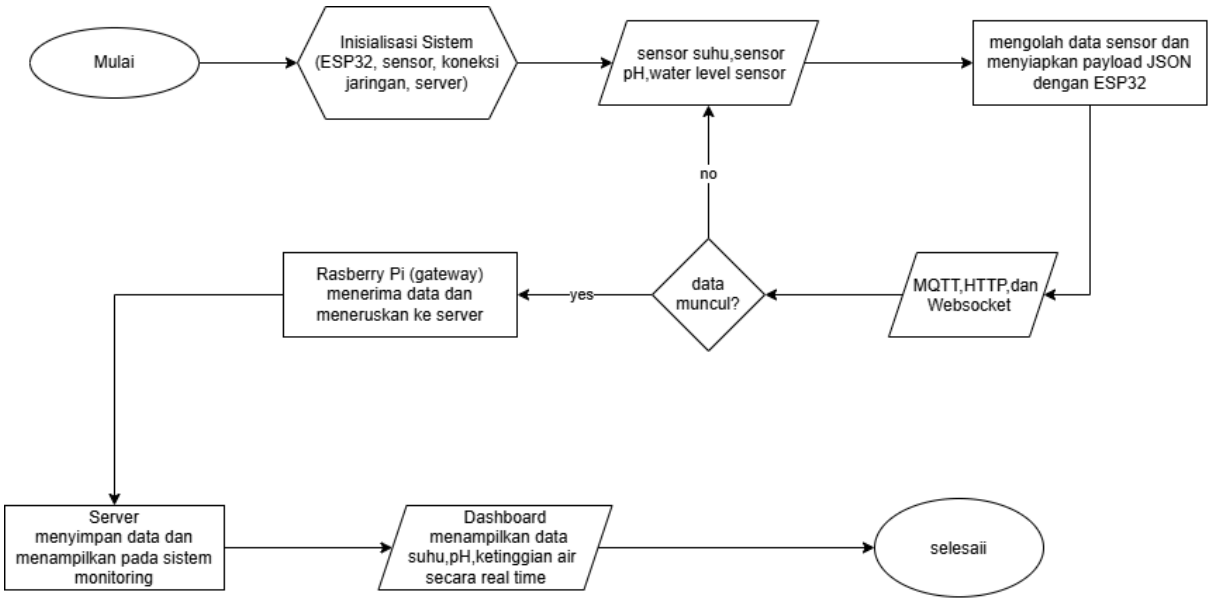
Gambar 6.4 Desain Topologi Server

Pada topologi sisi server tersebut, arsitektur jaringan terdiri dari sebuah router, sebuah Raspberry Pi sebagai Linux *Gateway*, dan sebuah sistem komputasi server utama. Untuk jalur komunikasinya, transmisi data nirkabel dari perangkat IoT akan diterima oleh router, lalu diteruskan menuju Raspberry Pi yang bertindak sebagai gerbang jaringan perantara.

Selain itu, paket data yang telah melewati gateway akan terkoneksi langsung menuju layanan penerima di server. Sedangkan untuk pemrosesan datanya sendiri yaitu menggunakan MQTT Broker, HTTP Server, dan WebSocket Server yang seluruh hasilnya disimpan secara terpusat pada sebuah Database, untuk kemudian dikelola oleh Web Server agar dapat ditampilkan pada layar *Smartphone* milik *User*.

6.4 Alur Kerja Sistem

Alur kerja sistem project tersebut adalah sebagai berikut:



Gambar 6.5 Flowchart Alur Kerja Sistem

6.5 Implementasi Protokol Komunikasi

Implementasi protokol komunikasi pada node IoT berbasis ESP32 dirancang dengan mempertahankan kesetaraan variabel untuk menjamin validitas pengujian komparatif [14]. Ketiga protokol mentransmisikan *payload* JSON yang identik menuju server yang sama.

6.5.1 MQTT

Protokol MQTT diimplementasikan menggunakan mekanisme *publish-subscribe* dengan memanfaatkan *broker* Mosquitto sebagai perantara pengiriman pesan antara node IoT dan server [1], [12]. MQTT dirancang sebagai protokol ringan (*lightweight*) yang berjalan di atas lapisan *Transmission Control Protocol* (TCP), sehingga direkomendasikan untuk perangkat dengan sumber daya terbatas dan jaringan berkapasitas rendah [14]. Dalam menjamin keandalan pengiriman pesan, MQTT menyediakan tiga tingkat *Quality of Service* (QoS), yaitu QoS 0, QoS 1, dan QoS 2 [4], [12]. Meskipun QoS 1 umumnya digunakan untuk memastikan pesan diterima melalui pengiriman *acknowledgement* (ACK) [12], implementasi MQTT pada eksperimen komparatif ini secara khusus dikunci pada QoS 0 (*At most once*). Penyesuaian ini dilakukan untuk menjamin asas keadilan eksperimen (*fairness*), di mana proses pemulihan kehilangan paket murni dikendalikan oleh mekanisme

transmisi ulang (*retransmission*) pada lapisan TCP, setara dengan perilaku bawaan pada HTTP dan WebSocket [7].

6.5.2 HTTP

Implementasi HTTP menggunakan pola komunikasi *request-response* berbasis *client-server*, di mana ESP32 bertindak sebagai *client* yang mengirimkan *request* dan server memberikan *response* [5], [12]. Pola komunikasi HTTP membawa *overhead* yang relatif besar akibat keharusan menyertakan *header* yang lengkap pada setiap *request* yang dikirimkan [5], [14]. Arsitektur ini memicu latensi yang tinggi akibat siklus pembentukan koneksi secara terus-menerus [11]. Untuk menghindari bias eksperimen berupa *overhead* ekstrem dari proses *3-way handshake* TCP, implementasi HTTP dikonfigurasi menggunakan standar HTTP/1.1 yang memanfaatkan *header* Connection: keep-alive guna mempertahankan sesi TCP tetap terbuka [5], [7].

6.5.3 WebSocket

WebSocket diimplementasikan untuk menyediakan kanal komunikasi *full-duplex* yang persisten di atas satu koneksi TCP tunggal [6], [14]. Komunikasi diawali dengan mekanisme *upgrade handshake* dari HTTP, yang selanjutnya memungkinkan transmisi data secara langsung dari dua arah dengan latensi yang sangat rendah [6], [14]. Melalui koneksi persisten ini, pengiriman data periodik tidak lagi memerlukan *header* pembuka yang besar seperti HTTP, melainkan hanya menyertakan *overhead frame* berukuran minimal (berkisar 2-10 *bytes*) untuk setiap paket data aplikasi [6], [14].

6.6 Perancangan Network Impairment

Perancangan *network impairment* (rekayasa kondisi jaringan) dilakukan untuk mensimulasikan dinamika lingkungan jaringan nyata yang bervariasi, guna menguji ketahanan serta karakteristik performa protokol MQTT, HTTP, dan WebSocket secara objektif. Simulasi gangguan ini tidak diterapkan langsung pada perangkat mikrokontroler, melainkan diisolasi dan dikendalikan melalui perangkat Raspberry Pi yang bertindak sebagai *Linux Gateway* yang berada di antara router dan server.

Di dalam sistem operasi Linux pada Raspberry Pi, perancangan ini memanfaatkan utilitas bawaan *kernel* yaitu tc (*Traffic Control*) dan netem (*Network Emulator*). Pengujian dibagi menjadi beberapa skenario variabel gangguan yang dikonfigurasi secara terkontrol dan terpisah untuk setiap sesi eksperimen:

a. Kondisi Normal (*Baseline* / S0): Pengujian awal tanpa penambahan gangguan buatan untuk mengukur performa dasar (*native performance*) dari masing-masing protokol pada jalur jaringan lokal yang ideal.
b. Simulasi *Latency* (Penundaan Jaringan): Menambahkan waktu tunggu buatan pada antrean paket data yang melintas di *gateway* (misalnya penambahan *delay* konstan sebesar 100 ms dan 300 ms) untuk mensimulasikan jarak server yang lebih jauh atau respons jaringan yang lambat.
c. Simulasi *Packet Loss* (Kehilangan Paket): Menyuntikkan persentase kehilangan paket secara acak pada lapisan jaringan (misalnya *loss* sebesar 1%, 5%, dan 10%) guna menguji reaktivitas protokol dalam menghadapi transmisi yang tidak stabil.
d. Simulasi *Bandwidth Limitation* (Pembatasan Kapasitas): Membatasi laju maksimum transmisi data pada *gateway* (misalnya dibatasi menjadi 1 Mbps atau 512 kbps) untuk mengamati perilaku protokol saat kapasitas pipa jaringan menyempit.

Melalui perancangan *network impairment* ini, setiap protokol (MQTT, HTTP, dan WebSocket) akan diuji di bawah beban gangguan yang persis sama, sehingga perbandingan parameter QoS seperti *delay*, *jitter*, *packet loss*, dan *throughput* yang dihasilkan menjadi adil, valid, dan dapat direproduksi.

6.7 Variabel Pengujian

a. Variabel Bebas:

Variabel bebas dalam eksperimen ini adalah kondisi kualitas jaringan yang direkayasa secara terkontrol menggunakan perangkat *Linux Gateway* [7]. Variabel ini meliputi:

1. Kondisi jaringan normal (*baseline*) [7].
2. Peningkatan nilai *latency* (contoh: +100 ms, +300 ms) [7].
3. Penambahan tingkat *packet loss* (contoh: 1%, 5%, 10%) [7].

4. Pembatasan kapasitas *bandwidth* maksimum (contoh: 1 Mbps, 512 kbps) [7].

b. Variabel Terikat

Variabel terikat merupakan parameter *Quality of Service* (QoS) jaringan dan metrik efisiensi komunikasi yang diekstraksi dari hasil pengujian [7], [14]. Parameter tersebut meliputi:

1. *Throughput:* Laju transmisi data yang berhasil dikirimkan oleh sistem per satuan waktu [14].

$$Throughput = \frac{Jumlah\ data\ diterima}{Waktu\ Pengamatan}$$

Kategori *Throughput* menurut standar TIPHON

Kategori	***Throughtput*(bps)**	**Indeks**
Sangat Baik	>95	4
Baik	75 – 95	3
Sedang	50 – 75	2
Buruk	<50	1

Tabel 6.1 Perhitungan Indeks Throughput

2. *Delay:* Waktu yang dibutuhkan untuk mentransmisikan paket data dari pengirim hingga tiba di titik penerima [12].

$$Delay = \frac{Total\ Delay}{Jumlah\ paket}$$

Kategori *Delay* menurut standar TIPHON

Kategori	***Delay*(ms)**	**Indeks**
Sangat Baik	<150	4
Baik	150 – 300	3
Sedang	300 – 450	2
Buruk	>450	1

Tabel 6.2 Perhitungan Indeks Delay

3. *Jitter:* Variasi *delay* antar-paket yang merepresentasikan tingkat fluktuasi waktu kedatangan data [14].

$$Jitter = \frac{Total\ variasi\ delay}{Total\ paket\ diterima}$$

Kategori *Jitter* menurut standar TIPHON

Kategori	***Jitter*(ms)**	**Indeks**
Sangat Baik	0	4
Baik	0 – 75	3
Sedang	75 – 125	2
Buruk	125 – 225	1

Tabel 6.3 Perhitungan Indeks Jitter

4. *Packet Loss* / *Packet Reception Ratio* (PRR): Persentase rasio jumlah paket data yang berhasil diterima terhadap total jumlah paket yang dikirimkan [12].

$$Packet\ Loss(\%) = \frac{Paket\ Dikirim - Paket\ Diterima}{Paket\ Dikirim} \text{ x } 100\%$$

Kategori *Packet Loss* menurut standar TIPHON

Kategori	***Packet Loss*(%)**	**Indeks**
Sangat Baik	0	4
Baik	3	3
Sedang	15	2
Buruk	25	1

Tabel 6.4 Perhitungan Indeks Packet Loss

5. *Response Time*: Waktu yang dibutuhkan sejak *request* atau pesan dikirim hingga *response* atau balasan diterima kembali oleh perangkat pengirim.

$$Response\ Time(\text{ms}) = \text{Waktu } Response \text{ diterima - Waktu } Request \text{ dikirim}$$

6. *Goodput*: Laju *useful payload* atau data aplikasi yang berguna dan berhasil diterima oleh perangkat tujuan dalam satuan waktu tertentu.

$$Goodput(\text{bps}) = \frac{Useful\ Payload(byte) \times 8}{Waktu(s)}$$

7. *Protocol Overhead*: Persentase data tambahan di luar *useful payload* yang diperlukan selama proses komunikasi untuk mendukung pengiriman data.

$$Protocol\ Overhead(\%) = \frac{Total\ data\ ditransmisikan - Useful\ Paylaod}{Total\ data\ ditransmisikan} \text{ x } 100\%$$

c. Variabel Kontrol

1. Pengujian menggunakan perangkat keras mikrokontroler yang identik (ESP32) untuk mengeliminasi bias perbedaan kemampuan pemrosesan *hardware* [14], [11].
2. Ketiga protokol diuji menggunakan interval waktu pengiriman paket data yang disamakan secara sistematis [14].
3. Seluruh protokol menggunakan ukuran (*byte*) dan format struktur data *payload* (JSON) yang sama [14].
4. Lingkungan perutean (*routing*), tipe router, perangkat server, dan titik penyadapan (*packet capture point*) dipertahankan agar tidak berubah selama tahapan pengujian [7].

6.8 Rencana Implementasi

Implementasi dimulai dengan menyiapkan perangkat keras, seperti ESP32 sebagai node IoT, sensor suhu DS18B20, sensor pH, water level sensor, router, Linux Gateway, dan server. ESP32 mengambil data dari ketiga sensor dan mengirimkannya melalui Wi-Fi ke server.

Pada sisi node, program ESP32 dikembangkan dengan Arduino IDE atau PlatformIO. Pada sisi server, disiapkan MQTT Broker (Mosquitto), HTTP Server, WebSocket Server, basis data untuk menyimpan hasil sensor dan parameter pengujian, serta dashboard untuk monitoring.

Ketiga protokol, MQTT, HTTP, dan WebSocket dapat diimplementasikan pada ESP32 dengan payload JSON identik, ukuran payload, interval pengiriman, dan jumlah pesan yang sama. MQTT dikunci pada QoS 0 dengan mekanisme *publish subscribe* melalui Mosquitto. HTTP menggunakan HTTP/1.1 dengan koneksi keep-alive. WebSocket menggunakan koneksi TCP persisten full-duplex.

Linux Gateway dikonfigurasi dengan traffic control dan netem untuk membentuk kondisi jaringan normal dan degradasi terkendali,seperti penambahan latency, packet loss, dan pembatasan bandwidth. Setiap kondisi direkam dengan Wireshark.

Sebelum pengujian utama, seluruh parameter komunikasi diverifikasi ulang,seperti perangkat keras, format payload, interval pengiriman, routing, router, server, dan titik packet capture untuk memastikan tidak ada perbedaan konfigurasi yang mencemari hasil perbandingan.

6.9 Rencana Pengujian

Pengujian berjalan dalam dua tahap. Tahap pertama adalah baseline,yaitu pengiriman data sensor dari ESP32 ke server pada jaringan normal, direkam dengan Wireshark sebagai acuan performa dasar.

Tahap kedua menguji kondisi jaringan yang direkayasa lewat Linux Gateway, satu variabel degradasi per pengujian,seperti latency, packet loss, atau bandwidth, tidak digabung. Ketiga protokol diuji pada setiap kondisi dengan parameter komunikasi tetap. Setiap sesi menghasilkan arsip PCAP untuk analisis throughput, delay, jitter, packet loss, response time, goodput, dan protocol overhead.

6.10 Skenario Pengujian

Skenario pengujian disusun berdasarkan variabel bebas berupa kondisi kualitas jaringan yang direkayasa secara terkontrol. Pengujian dimulai dari kondisi jaringan normal sebagai baseline, kemudian dilanjutkan dengan pengujian peningkatan latency,

packet loss, dan pembatasan bandwidth secara terpisah. Pemisahan setiap jenis network impairment dilakukan untuk mengetahui pengaruh masing-masing variabel terhadap performa protokol.

A. Uji Tanpa Impairment dan Tanpa AES

No	Parameter	Parameter Rekayasa	Protokol yang Diuji
1	Baseline	Jaringan normal tanpa *impairment* dan tanpa enkripsi AES	MQTT, HTTP, WebSocket

Tabel 6.5 Skenario Pengujian Tanpa Impairment dan Tanpa AES

B. Uji Network Impairment Tanpa AES

No	Kondisi Jaringan	Parameter Rekayasa	Protokol yang Diuji
2	Latency 1	Penambahan latency +100 ms	MQTT, HTTP, WebSocket
3	Latency 2	Penambahan latency +200 ms	MQTT, HTTP, WebSocket
4	Latency 3	Penambahan latency +300 ms	MQTT, HTTP, WebSocket
5	Latency 4	Penambahan latency +400 ms	MQTT, HTTP, WebSocket
6	Latency 5	Penambahan latency +500 ms	MQTT, HTTP, WebSocket
7	Latency 6	Penambahan latency +600 ms	MQTT, HTTP, WebSocket
8	Latency 7	Penambahan latency +700 ms	MQTT, HTTP, WebSocket
9	Latency 8	Penambahan latency +800 ms	MQTT, HTTP, WebSocket
10	Latency 9	Penambahan latency +900 ms	MQTT, HTTP, WebSocket
11	Latency 10	Penambahan latency +1000 ms	MQTT, HTTP, WebSocket

No	Kondisi Jaringan	Parameter Rekayasa	Protokol yang Diuji
12	Packet Loss 1	Packet loss 10%	MQTT, HTTP, WebSocket
13	Packet Loss 2	Packet loss 20%	MQTT, HTTP, WebSocket
14	Packet Loss 3	Packet loss 30%	MQTT, HTTP, WebSocket
15	Packet Loss 4	Packet loss 40%	MQTT, HTTP, WebSocket
16	Packet Loss 5	Packet loss 50%	MQTT, HTTP, WebSocket
17	Packet Loss 6	Packet loss 60%	MQTT, HTTP, WebSocket
18	Packet Loss 7	Packet loss 70%	MQTT, HTTP, WebSocket
19	Packet Loss 8	Packet loss 80%	MQTT, HTTP, WebSocket
20	Packet Loss 9	Packet loss 90%	MQTT, HTTP, WebSocket
21	Packet Loss 10	Packet loss 100%	MQTT, HTTP, WebSocket
22	Bandwidth 1	Pembatasan bandwidth 1024 Kbps	MQTT, HTTP, WebSocket
23	Bandwidth 2	Pembatasan bandwidth 512 Kbps	MQTT, HTTP, WebSocket
24	Bandwidth 3	Pembatasan bandwidth 128 Kbps	MQTT, HTTP, WebSocket
25	Bandwidth 4	Pembatasan bandwidth 64 Kbps	MQTT, HTTP, WebSocket
26	Bandwidth 5	Pembatasan bandwidth 32 Kbps	MQTT, HTTP, WebSocket
27	Bandwidth 6	Pembatasan bandwidth 16 Kbps	MQTT, HTTP, WebSocket
28	Bandwidth 7	Pembatasan bandwidth 8 Kbps	MQTT, HTTP,

No	Kondisi Jaringan	Parameter Rekayasa	Protokol yang Diuji
			WebSocket
29	Bandwidth 8	Pembatasan bandwidth 4 Kbps	MQTT, HTTP, WebSocket
30	Bandwidth 9	Pembatasan bandwidth 2 Kbps	MQTT, HTTP, WebSocket
31	Bandwidth 10	Pembatasan bandwidth 1 Kbps	MQTT, HTTP, WebSocket

Tabel 6.6 Skenario Pengujian Tanpa Impairment dan Tanpa AES

C. Uji Network Impairment dan AES

No	Kondisi Jaringan	Parameter Rekayasa	Protokol yang Diuji
2	Latency 1	Penambahan latency +100 ms	MQTT, HTTP, WebSocket
3	Latency 2	Penambahan latency +200 ms	MQTT, HTTP, WebSocket
4	Latency 3	Penambahan latency +300 ms	MQTT, HTTP, WebSocket
5	Latency 4	Penambahan latency +400 ms	MQTT, HTTP, WebSocket
6	Latency 5	Penambahan latency +500 ms	MQTT, HTTP, WebSocket
7	Latency 6	Penambahan latency +600 ms	MQTT, HTTP, WebSocket
8	Latency 7	Penambahan latency +700 ms	MQTT, HTTP, WebSocket
9	Latency 8	Penambahan latency +800 ms	MQTT, HTTP, WebSocket
10	Latency 9	Penambahan latency +900 ms	MQTT, HTTP, WebSocket
11	Latency 10	Penambahan latency +1000 ms	MQTT, HTTP, WebSocket

No	Kondisi Jaringan	Parameter Rekayasa	Protokol yang Diuji
12	Packet Loss 1	Packet loss 10%	MQTT, HTTP, WebSocket
13	Packet Loss 2	Packet loss 20%	MQTT, HTTP, WebSocket
14	Packet Loss 3	Packet loss 30%	MQTT, HTTP, WebSocket
15	Packet Loss 4	Packet loss 40%	MQTT, HTTP, WebSocket
16	Packet Loss 5	Packet loss 50%	MQTT, HTTP, WebSocket
17	Packet Loss 6	Packet loss 60%	MQTT, HTTP, WebSocket
18	Packet Loss 7	Packet loss 70%	MQTT, HTTP, WebSocket
19	Packet Loss 8	Packet loss 80%	MQTT, HTTP, WebSocket
20	Packet Loss 9	Packet loss 90%	MQTT, HTTP, WebSocket
21	Packet Loss 10	Packet loss 100%	MQTT, HTTP, WebSocket
22	Bandwidth 1	Pembatasan bandwidth 1024 Kbps	MQTT, HTTP, WebSocket
23	Bandwidth 2	Pembatasan bandwidth 512 Kbps	MQTT, HTTP, WebSocket
24	Bandwidth 3	Pembatasan bandwidth 128 Kbps	MQTT, HTTP, WebSocket
25	Bandwidth 4	Pembatasan bandwidth 64 Kbps	MQTT, HTTP, WebSocket
26	Bandwidth 5	Pembatasan bandwidth 32 Kbps	MQTT, HTTP, WebSocket
27	Bandwidth 6	Pembatasan bandwidth 16 Kbps	MQTT, HTTP, WebSocket
28	Bandwidth 7	Pembatasan bandwidth 8 Kbps	MQTT, HTTP,

No	Kondisi Jaringan	Parameter Rekayasa	Protokol yang Diuji
			WebSocket
29	Bandwidth 8	Pembatasan bandwidth 4 Kbps	MQTT, HTTP, WebSocket
30	Bandwidth 9	Pembatasan bandwidth 2 Kbps	MQTT, HTTP, WebSocket
31	Bandwidth 10	Pembatasan bandwidth 1 Kbps	MQTT, HTTP, WebSocket

Tabel 6.7 Skenario Pengujian Dengan Impairment dan AES

D. Uji AES dan Tanpa Network Impairment

No	Parameter	Parameter Rekayasa	Protokol yang Diuji
1	Baseline	Jaringan normal tanpa *impairment* namun dengan enkripsi AES	MQTT, HTTP, WebSocket

Tabel 6.8 Skenario Pengujian AES dan Tanpa Network Impairment

Metode Analisis Hasil

Arsip PCAP diproses dengan Wireshark untuk mengekstrak parameter performa: throughput (laju transmisi berhasil per satuan waktu), delay (waktu pengiriman ke penerimaan), jitter (variasi delay antar-paket), packet loss atau Packet Reception Ratio, response time (dari permintaan hingga balasan), protocol overhead (proporsi kontrol protokol terhadap data aplikasi), dan goodput (payload berguna per waktu transmisi).

Nilai-nilai ini dibandingkan antar-protokol di seluruh skenario untuk melihat bagaimana peningkatan latency, packet loss, dan pembatasan bandwidth mengubah karakteristik masing-masing protokol. Tujuannya bukan menetapkan protokol "terbaik" secara universal, melainkan memetakan *trade-off* tiap protokol pada kondisi jaringan tertentu.

6.11 Proses, Luaran, Indikator Capaian, dan Penanggung Jawab

Proyek berjalan dalam delapan tahap berurutan,seperti studi literatur dan analisis kebutuhan, perancangan sistem, implementasi node IoT dan protokol komunikasi, konfigurasi lingkungan jaringan, pengujian, packet capture, analisis hasil, dan penyusunan laporan. Setiap tahap punya luaran dan indikator capaian yang menunjukkan kesesuaian pelaksanaan dengan rancangan proyek.

Metodologi ini memakai ESP32 sebagai node, tiga sensor (suhu, pH, water level), tiga protokol komunikasi (MQTT, HTTP, WebSocket), Linux Gateway dengan network impairment, dan Wireshark untuk pengambilan paket.

No	Proses	Luaran	Indikator Capaian	Penanggung Jawab
1.	Studi literatur dan analisis kebutuhan	Daftar kebutuhan perangkat keras, perangkat lunak, protokol, dan parameter pengujian	Kebutuhan sistem dan testbed jaringan telah teridentifikasi sesuai ruang lingkup proyek	Seluruh anggota
2.	Perancangan arsitektur sistem dan testbed jaringan	Rancangan topologi dan alur komunikasi ESP32, router, Linux Gateway, dan server	Posisi ESP32, Router, Linux Gateway, server, dan packet capture point telah ditentukan	Seluruh anggota
3.	Implementasi node IoT	Prototype node ESP32 dan sensor kualitas air	ESP32 mampu memperoleh dan menyiapkan data dari ketiga sensor	Aris Nur Rochman
4.	Implementasi MQTT, HTTP, dan WebSocket	Program komunikasi pada ESP32 dan layanan penerima pada server	Ketiga protokol dapat digunakan untuk mengirim dan menerima payload JSON pada server yang sama	Daniel Martin Gideon Siregar
5.	Konfigurasi server, database, dan dashboard	Server penerima, penyimpanan data, dan dashboard monitoring	Data dari ketiga protokol dapat diterima, diproses, disimpan, dan ditampilkan	Muhammad Rafli Saputra
6.	Konfigurasi Linux Gateway	Gateway pengujian dengan konfigurasi	Kondisi baseline, latency, packet loss,	Daniel Martin Gideon Siregar

No	Proses	Luaran	Indikator Capaian	Penanggung Jawab
	dan network impairment	tc/netem	dan bandwidth limitation dapat diterapkan secara terkontrol	
7.	Kalibrasi parameter eksperimen	Parameter eksperimen yang telah diseragamkan	Payload, interval pengiriman, jumlah pesan, perangkat, jaringan, server, dan packet capture point telah disamakan	Seluruh anggota
8.	Pengujian baseline	Data packet capture kondisi jaringan normal	Tersedia arsip PCAP dan data performa dasar dari MQTT, HTTP, dan WebSocket	Seluruh anggota
9.	Pengujian skenario degradasi jaringan	Arsip PCAP setiap skenario pengujian	Ketiga protokol telah diuji pada setiap kondisi latency, packet loss, dan bandwidth yang ditentukan	Seluruh anggota
10.	Ekstraksi dan analisis data	Data throughput, delay, jitter, packet loss, response time, goodput, dan protocol overhead	Seluruh parameter performa berhasil diperoleh dari hasil packet capture dan dapat dibandingkan antara protokol	Muhammad Rafli Saputra
11.	Penyusunan hasil dan kesimpulan	Hasil analisis, pembahasan, kesimpulan, dan dokumentasi proyek	Tersusun hasil proyek yang menjelaskan karakteristik dan trade-off MQTT, HTTP, dan WebSocket	Seluruh anggota

Tabel 6.9 Proses Luaran dan Indikator

6.12 Pembagian Tugas Anggota

Pembagian tugas dilakukan berdasarkan komponen utama dalam pelaksanaan proyek, yaitu perangkat IoT, infrastruktur jaringan dan server, serta pengambilan dan analisis data. Tahap kalibrasi, pengujian komparatif, validasi konfigurasi, dan penyusunan hasil tetap dilakukan secara bersama karena ketiga protokol harus diuji dengan parameter yang sama agar kondisi eksperimen tetap setara.

No	Anggota	Pembagian Tugas
1.	Aris Nur Rochman	Bertanggung jawab pada perangkat IoT, meliputi ESP32, sensor suhu, sensor pH, dan water level sensor, serta membantu implementasi pengiriman data menggunakan MQTT, HTTP, dan WebSocket.
2.	Daniel Martin Gideon Siregar	Bertanggung jawab pada infrastruktur jaringan dan server, meliputi Router, Linux Gateway/Raspberry Pi, konfigurasi tc/netem, network impairment, serta layanan server dan penyimpanan data.
3.	Muhammad Rafli Saputra	Bertanggung jawab pada packet capture dan analisis data menggunakan Wireshark, pengolahan hasil pengujian, perbandingan parameter performa, serta dokumentasi hasil proyek.
4.	Seluruh Anggota	Melakukan kalibrasi parameter eksperimen, memastikan kesetaraan payload, interval pengiriman, jumlah pesan, perangkat, jaringan, server, dan packet capture point, serta melaksanakan pengujian dan penyusunan laporan secara bersama.

Tabel 6.10 Pembagian Tugas Anggota

7. RENCANA KEGIATAN & TIMELINE

Rencana kegiatan proyek disusun selama 16 minggu yang mencakup tahap analisis kebutuhan, studi literatur, perancangan sistem, penyusunan proposal, implementasi perangkat dan jaringan, pengujian, evaluasi hasil, hingga penyusunan laporan dan presentasi akhir.

No.	Kegiatan	Minggu															
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	Analisis kebutuhan	●	●														
2	Studi literatur		●	●	●												
3	Desain topologi dan arsitektur testbed			●	●	●											
4	Konsultasi Proposal PBL				●	●	●	●									

No.	Kegiatan	Minggu															
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
5	Draft dan revisi Proposal PBL						●	●									
6	Presentasi Proposal PBL							●									
7	Integrasi jaringan dan Linux Gateway								●	●	●						
8	Integrasi ESP32 dan sensor dengan Raspberry Pi Gateway									●	●						
9	Implementasi protokol komunikasi, server, dan dashboard									●	●	●	●	●			
10	Pengujian jaringan dan QoS											●	●	●	●		
11	Evaluasi dan analisis hasil pengujian												●	●	●		
12	Dokumentasi dan laporan													●	●	●	
13	Presentasi akhir PBL															●	●

Tabel 7.1 Rencana Kegiatan dan Timeline

8. DELIVERABLES / LUARAN

Luaran yang dihasilkan dari proyek “Analisis Performa Protokol Komunikasi pada Sistem Monitoring Kolam Lele Berbasis IoT” meliputi:

No.	Luaran	Bentuk Luaran
1	Prototipe IoT Kolam Lele	Sistem sensor, ESP32, jaringan, gateway, dan server
2	Website Dashboard Monitoring	Aplikasi web monitoring data sensor
3	Implementasi MQTT, HTTP, dan WebSocket	Sistem komunikasi

No.	Luaran	Bentuk Luaran
		ESP32–server
4	Network Impairment Testbed	Linux Gateway dengan tc/netem
5	Dataset Pengujian	Data hasil seluruh skenario eksperimen
6	Hasil Analisis QoS	Perbandingan *delay, jitter, packet loss*, dan *throughput*
7	Hasil Analisis Efisiensi	Response time, goodput, dan protocol overhead
8	Packet Capture	File PCAP MQTT, HTTP, dan WebSocket
9	Source Code & Konfigurasi	Firmware, backend, dashboard, gateway
10	Laporan Akhir PBL	Dokumentasi dan hasil proyek

Tabel 8.1 Luaran Project

9. ESTIMASI ANGGARAN

Estimasi anggaran proyek disusun berdasarkan kebutuhan perangkat keras dan komponen pendukung yang digunakan dalam proses implementasi sistem monitoring kolam lele berbasis IoT. Sebagian besar perangkat direncanakan menggunakan fasilitas atau peminjaman alat laboratorium sehingga tidak menimbulkan biaya pengadaan langsung.

Uraian	Pinjam	Beli	Vol. 1	Sat. 1	Vol. 2	Sat. 2	Harga Satuan	Satuan Akhir	Jumlah (Total Rp)
Belanja Barang (Peminjaman Alat Laboratorium)									
ESP32	☑	☐	1	LS	3	PCS	Rp88.000	LS	Rp0
MikroTik hAP ac lite	☑	☐	1	LS	1	PCS	Rp1.150.000	LS	Rp0
Sensor pH PH-4502C	☐	☑	1	LS	3	PCS	Rp284.000	LS	Rp0
Sensor Suhu DS18B20	☐	☑	1	LS	3	PCS	Rp42.000	LS	Rp0
Sensor Ketinggian Air	☐	☑	1	LS	3	PCS	Rp5000	LS	Rp0
Raspberry Pi + Media Penyimpanan 128 GB	☑	☐	1	LS	1	SET	Rp4.098.000	LS	Rp0
Kabel LAN RJ45 ±2 Meter	☐	☑	1	LS	6	PCS	Rp6000	LS	Rp0

Breadboard	☑	☐	1	LS	3	PCS	Rp8.500	LS	Rp0
Kabel Tunggal Male-to-Male ±3 Meter	☑	☐	1	LS	1	SET	Rp5000	LS	Rp0
Kabel Jumper Male-to-Female	☑	☐	1	LS	20	PCS	Rp5000	LS	Rp0
Case Raspberry Pi	☐	☐	1	LS	1	PCS	Rp18.500	LS	Rp0
Power Supply 12 V	☑	☐	1	LS	1	PCS	Rp50.000	LS	Rp0
Step Down DC-to-DC Adjustable	☑	☐	1	LS	1	PCS	Rp30.500	LS	Rp0
Total Anggaran							5.853.500		Rp0

Tabel 9.1 Daftar dan Estimasi Anggaran

10. DAFTAR PUSTAKA

[1] E. K. Pramartaningthyas, M. Hariyadi, N. Afiyat, S. Ma'shumah, dan D. Sigiono, "Evaluasi Kinerja Sistem IoT untuk Monitoring pH dan Kontrol Suhu Air pada Akuarium Benih Ikan Clarias gariepinus," *Infotekmesin*, vol. 17, no. 2, 2026.

[2] R. A. N. Yuusufa, N. H. Hari, Y. D. P. Negara, dan K. Umam, "Web-Based Monitoring and Control System for Aquaponics Using IoT and Fuzzy Logic," *Journal of Advanced Vocational Information and Communication Technology*, vol. 1, no. 2, pp. 66–86, 2026.

[3] Ardiansyah dan F. Y. Pamuji, "Penerapan Standar TIPHON untuk Kinerja Quality of Service Jaringan Internet: Studi Kasus di Jaringan Alfanet," *JATI*, vol. 9, no. 5, 2025.

[4] S. O. F. Tarigan, H. I. Sitepu, dan M. Hutagalung, "Pengukuran Kinerja Sistem Publish/Subscribe Menggunakan Protokol MQTT (Message Queuing Telemetry Transport)," *Jurnal Telematika*, vol. 9, no. 1, 2014.

[5] M. I. Yamin, A. Ariman, R. M. Dinata, dan N. Purnomo, "Comparative Performance Evaluation of MQTT, WebSocket, and Firebase on ESP32-C3 for Real-Time IoT Communication," *JATI*, vol. 10, no. 3, 2026.

[6] B. Amirkhanov, G. Amirkhanova, M. Kunelbayev, S. Adilzhanova, dan M. Tokhtassyn, "Evaluating HTTP, MQTT over TCP and MQTT over WebSocket for Digital Twin Applications: A Comparative Analysis on Latency, Stability, and Integration,"

International Journal of Innovative Research and Scientific Studies, vol. 8, no. 1, pp. 679–694, 2025.

[7] J. F. Sakinah, “Implementasi dan Analisis Perbandingan Quality of Service Protokol Komunikasi IoT pada Sistem Kontrol dan Pemantauan Smart Pond,” Skripsi D4, Politeknik Negeri Jakarta, 2024.

[8] R. Nugroho, A. I. Pradana, dan B. W. Pamekas, “Analisis Komparatif QoS MQTT dan CoAP Berbasis ESP32 pada Platform ThingsBoard dalam Kondisi Jaringan Intermiten,” *JATI (Jurnal Mahasiswa Teknik Informatika)*, vol. 10, no. 4, 2026.

[9] Y. Savchenko dan I. Klymenko, “Factor-Wise Analysis of MQTT QoS Mechanism Impact on Quality-of-Service Metrics in Edge-Oriented IoT Networks,” *The International Conference on Security, Fault Tolerance, Intelligence*, 2026.

[10] R. A. Siregar dan F. Riza, "Sistem Kontrol Pergantian Air Pada Kolam Budidaya Ikan Lele Berbasis Internet Of Things," *PRINSIP: Portal Riset & Inovasi Sistem Perangkat Lunak*, vol. 4, no. 1, pp. 84-93, Jan. 2026.

[11] S. Adriansyah, Sumarno, U. M. Nurfuha, A. Giffari, T. Khoirunisa, dan W. G. Subarkah, "Performance Analysis of MQTT and HTTP Protocols on Low-Power ESP32 Devices for IoT Applications," *CoreID Journal*, vol. 4, no. 1, pp. 38-45, Mar. 2026.

[12] F. A. Fauzi, M. K. Delimayanti, dan V. Frendiana, "Analisis Implementasi Protokol MQTT dan HTTP pada Sistem Monitoring Berbasis IoT," dalam *Proc. Seminar Nasional Inovasi Vokasi (SNIV)*, vol. 5, no. 1, pp. 197-205, Jun. 2026.

[13] T. Widodo, A. B. Santoso, S. I. I. Ishak, dan R. Rumeon, "Sistem Kendali Proporsional Kualitas Air berupa Ph dan Suhu pada Budidaya Ikan Lele Berbasis IoT," *JEPIN (Jurnal Edukasi dan Penelitian Informatika)*, vol. 9, no. 1, pp. 59-66, Apr. 2023.

[14] S. Aulia dan S. Mujiyono, "Analisis QoS Protokol HTTP, MQTT, dan WebSocket Pada IoT Berbasis ESP8266," *JIPI (Jurnal Ilmiah Penelitian dan Pembelajaran Informatika)*, vol. 11, no. 3, pp. 1900-1911, Sep. 2026.